CSE 207 • CT-1 RECOVERY NOTES

The Missing Bridge

Lectures 03, 04 & 05: Prim's MST, shortest-path foundations, and Bellman–Ford

> **Professor-mode promise:** every algorithm is tied to its invariant, every proof has an exam-ready skeleton, and every dry run shows exactly what must be written for method marks.

Reconstructed from the progression of Lectures 01, 02, 06, 07 and 08 • Aligned to CLRS, 3rd ed., Chapters 23–24

What was actually missing?

Evidence from available material	Consequence
Lecture 02 finishes Kruskal and announces the "second strategy."	Lecture 03 must begin with **Prim's algorithm**.
Lecture 06 opens with a Bellman–Ford simulation and then proves it.	Lectures 04–05 must introduce **SSSP, relaxation, and Bellman–Ford**.
Lecture 07 starts with DAG shortest paths; Lecture 08 covers Dijkstra.	DAG-SP and Dijkstra are later material, not the main missing bridge.

LECTURE 03
Prim; safe edge; key and parent; priority queue; correctness; complexity; Prim vs Kruskal.

LECTURE 04
Shortest-path meaning; negative cycles; optimal substructure; initialization; relaxation; core properties.

LECTURE 05
Bellman–Ford; $|V|-1$ passes; dry run; reachable negative-cycle test; correctness; complexity.

The two questions that organize everything

MST: Which cheapest edge safely grows a tree? | **SSSP:** Can this edge improve a distance estimate?

Never merge these ideas. Prim stores the cheapest *single edge* attaching a vertex to the tree. Shortest-path algorithms store the cheapest *total path weight from source s*.

Notation used throughout

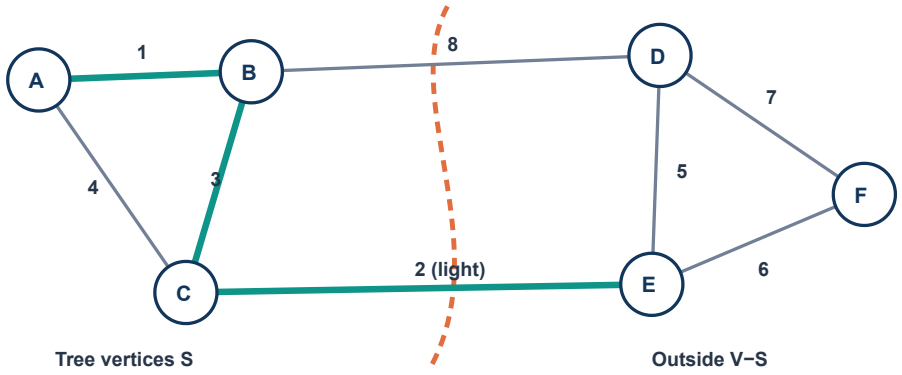
Symbol	Meaning	Symbol	Meaning
$G=(V,E)$	graph	$w(u,v)$	edge weight
key[v]	best attachment edge known in Prim	π[v]	parent / predecessor
$\delta(s,v)$	true shortest-path weight	d[v]	current upper-bound estimate

Study order: understand the pictures → reproduce pseudocode → trace tables → memorize proof skeletons → answer the practice set without looking.

LECTURE 03

Prim's algorithm: grow one connected tree

Prim begins at an arbitrary root r. At every step it chooses the minimum-weight edge crossing from the vertices already in the tree to a vertex outside it.



The current cut respects chosen edges. CE(2) is the light crossing edge, so it is safe.

The data stored

Item	Meaning	When changed?
key[v]	minimum weight of an edge currently known from the tree to v	when a newly added u offers w(u,v)<key[v]
π[v]	tree vertex giving that best edge	together with key[v]
Q	vertices not yet included in the MST	remove the minimum-key vertex each iteration

Invariant. At the start of each loop, V−Q is one connected partial tree, and for each v∈Q, key[v] is the lightest known edge joining v to V−Q.

Key is not a path distance. If A—B costs 4 and B—C costs 2, Prim may store key[C]=2, not 6. Prim minimizes the next attachment edge.

Prim: pseudocode and a complete dry run

```
MST-PRIM(G, w, r)
for each vertex u ∈ G.V
    u.key = ∞
    u.π = NIL
r.key = 0
Q = G.V                          // min-priority queue
while Q ≠ ∅
    u = EXTRACT-MIN(Q)
    for each v ∈ G.Adj[u]
        if v ∈ Q and w(u,v) < v.key
            v.π = u
            v.key = w(u,v)
            DECREASE-KEY(Q, v, w(u,v))
return { (v, v.π) : v ≠ r }
```

Worked graph

Edges: AB=1, AC=4, BC=3, BD=2, CD=7, CE=2, DE=5. Start at A.

Extract	Tree after extraction	Updates caused	Keys outside tree
A(0)	{A}	B←1 via A; C←4 via A	B:1, C:4, D:∞, E:∞
B(1)	{A,B}	C←3 via B; D←2 via B	D:2, C:3, E:∞
D(2)	{A,B,D}	E←5 via D; C stays 3	C:3, E:5
C(3)	{A,B,C,D}	E←2 via C	E:2
E(2)	all vertices	done	—

$$\text{MST edges} = \text{AB}(1), \text{BD}(2), \text{BC}(3), \text{CE}(2) \Rightarrow \text{total weight} = 8$$

HOW TO EARN DRY-RUN MARKS

Show extraction order, key changes, parent changes, final $V-1$ edges, and total weight. Merely drawing the final tree hides your method.

Why each chosen edge is safe

CUT-PROPERTY PROOF

Immediately before u is extracted, take the cut $(V-Q, Q)$. All already selected parent edges lie inside $V-Q$, so the cut respects A. Since u has minimum key in Q, $(u, \pi[u])$ is a light edge crossing this cut. By the cut theorem it is safe. Inductively every selected edge belongs to some MST; after $V-1$ selections the result is an MST.

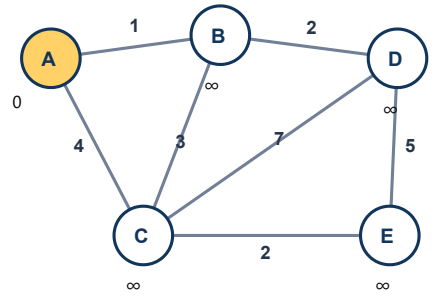
Complexity

Implementation	Time	Best interpretation
Adjacency matrix + linear scan	$\Theta(V^2)$	simple; often good for dense graphs
Adjacency list + binary heap	$O(E \log V)$	standard sparse-graph implementation
Adjacency list + Fibonacci heap	$O(E + V \log V)$ amortized	theoretical improvement when decrease-key dominates

Prim simulation I: initialization → A → B

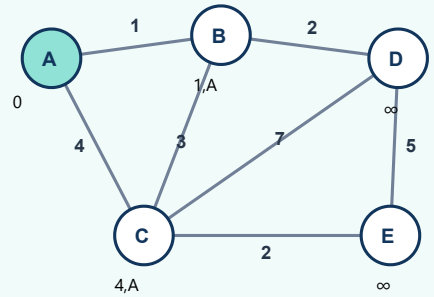
already in MST just extracted **green edge = chosen parent**

Frame 0 — initialize at A



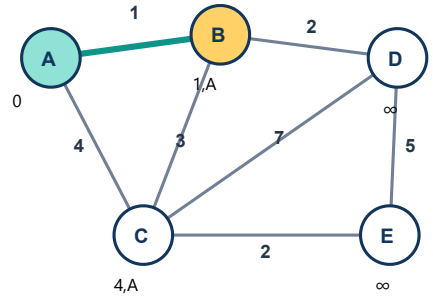
`key: A=0, B=∞, C=∞, D=∞, E=∞`

Frame 1 — extract A; scan A's edges



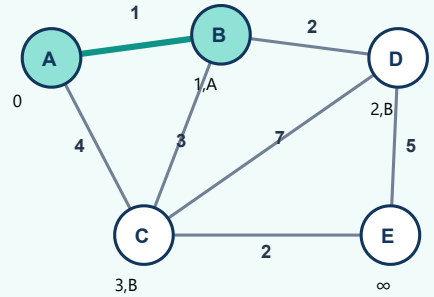
`B: ∞→1 via A; C: ∞→4 via A`

Frame 2 — extract B with minimum key 1



`choose parent edge AB(1); MST weight so far=1`

Frame 3 — scan B's outside neighbors

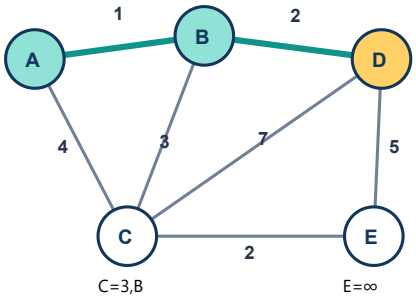


`C: 4→3 via B; D: ∞→2 via B`

Read each node label as "key,parent." The smallest key remaining is now D=2, so D is extracted next—not C.

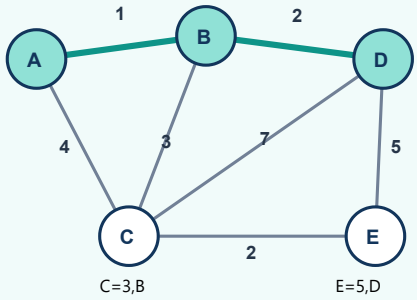
Prim simulation II: D → C → E

Frame 4 — extract D(2); choose BD



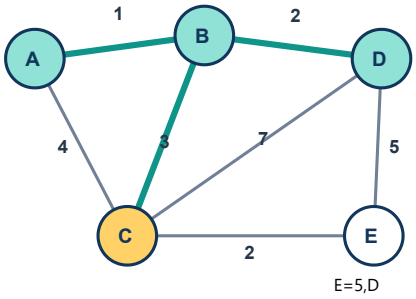
`MST={AB,BD}; weight=3`

Frame 5 — scan D; E becomes 5 via D



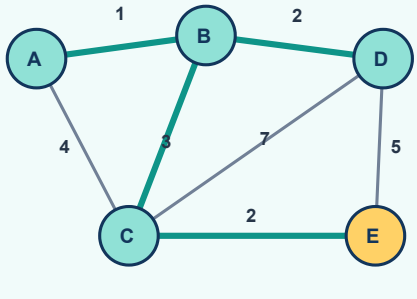
`CD=7 cannot beat C’s key 3; E: ∞→5`

Frame 6 — extract C(3); choose BC



`MST={AB,BD,BC}; weight=6`

Frame 7 — C improves E; extract E



`E: 5 via D → 2 via C; final weight=8`

Simulation conclusion: extraction order A,B,D,C,E. Parent array: π[B]=A, π[D]=B, π[C]=B, π[E]=C. The rejected edges are not “bad”; they simply never become the cheapest crossing attachment.

Prim vs Kruskal—and the traps examiners like

Feature	Prim	Kruskal
Growth	one connected tree	a forest; components merge
Greedy choice	minimum edge leaving current tree	globally next lightest edge that causes no cycle
Main structure	min-priority queue; key and parent	sorted edges; disjoint-set union
Typical time	O(E log V) with binary heap	O(E log E)=O(E log V)
Disconnected graph	restart per component for an MSF	naturally returns an MSF
Negative weights	allowed	allowed

WRONG CLAIM
"Prim always chooses the globally smallest unused edge."
Correction: it chooses the smallest edge crossing the current tree cut.

WRONG CLAIM
"The root changes the MST weight."
Correction: it may change which MST appears under ties, never the optimum weight.

WRONG CLAIM
"Distinct weights are required."
Correction: no; ties can create multiple valid MSTs.

WRONG CLAIM
"Prim is Dijkstra on an undirected graph."
Correction: they share a queue pattern but use different keys.

Exam-ready short answers

Why does an MST have V−1 edges? A spanning tree is connected and acyclic; every tree on V vertices has exactly V−1 edges.

Can an MST contain a negative edge? Yes. MST algorithms compare edge weights; negative weights create no "unbounded walk" issue because a tree has no cycle.

When is an MST unique? Distinct edge weights guarantee uniqueness. Equal weights do not necessarily imply non-uniqueness.

What is a safe edge? An edge whose addition to A preserves the existence of an MST containing $A \cup \{e\}$.

Memory line: Kruskal asks "Do the endpoints belong to different components?" Prim asks "Is this the cheapest door from my current room?"

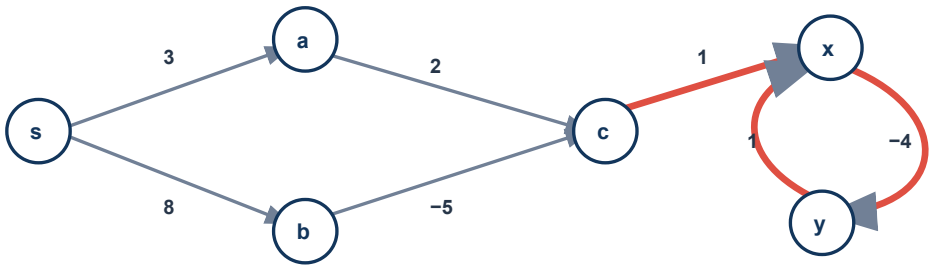
LECTURE 04

Single-source shortest paths: the mathematical ground

Given a weighted directed graph G=(V,E), weight function w:E→ℝ, and source s, SSSP asks for the minimum path weight from s to every vertex.

$$w(p)=\Sigma\, w(v_{i-1},v_i) \qquad \delta(s,v)=\text{minimum path weight from s to v}$$

Situation	δ(s,v)	Meaning
At least one path; no usable negative cycle	finite	a shortest path exists
No path from s to v	∞	v is unreachable
A reachable negative cycle can reach v	−∞	no finite shortest path exists



The x↔y cycle weighs −3. Because it is reachable from s, distances to x, y—and anything reachable from them—are −∞.

Optimal substructure

Lemma. Every subpath of a shortest path is itself a shortest path between its endpoints.
Proof by replacement: if a subpath were not shortest, replace it by a cheaper route; the entire s→v path becomes cheaper, contradicting shortestness.

Subtle point: negative edges are not themselves the problem. A *reachable negative cycle* is the obstruction. Bellman–Ford supports negative edges and detects such cycles.

Initialization and relaxation: the engine of SSSP

```
INITIALIZE-SINGLE-SOURCE(G, s)
for each vertex v ∈ G.V
    v.d = ∞
    v.π = NIL
s.d = 0

RELAX(u, v, w)
if v.d > u.d + w(u,v)
    v.d = u.d + w(u,v)
    v.π = u
```

Relax (u,v) exactly when d[v] > d[u] + w(u,v)

One edge, slowly

Before	Edge	Test	After
d[u]=7, d[v]=15	w(u,v)=3	15 > 7+3 ✓	d[v]=10, π[v]=u
d[u]=7, d[v]=9	w(u,v)=3	9 > 10 ✗	no change
d[u]=∞	any finite weight	∞+w=∞	unreachable u cannot improve v

Meaning of d[v]. It is not a guess below the truth. It is the weight of some discovered s→v walk, hence an **upper bound** on δ(s,v). Relaxation can only lower it.

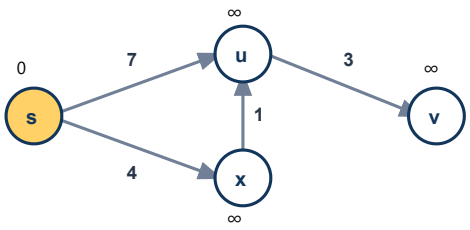
The properties worth quoting

Property	Exam-ready statement	Why it matters
Triangle inequality	δ(s,v) ≤ δ(s,u)+w(u,v)	the true best route to v is no worse than going optimally to u then taking (u,v)
Upper bound	d[v] ≥ δ(s,v) always; once equal, it never changes	estimates approach truth from above
No-path	if v is unreachable, d[v]=δ(s,v)=∞	no relaxation can invent a path
Convergence	if d[u]=δ(s,u), relaxing a shortest-path edge (u,v) makes d[v]=δ(s,v)	correctness propagates one edge forward
Path relaxation	relax shortest-path edges in order ⇒ final vertex becomes correct	core of Bellman–Ford and DAG-SP proofs
Predecessor subgraph	when all reachable distances are correct, π edges form a shortest-path tree	turns numbers into actual routes

Relaxation simulation: watch one improvement propagate

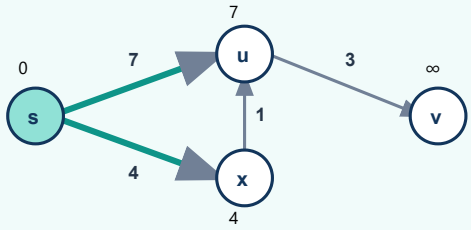
Graph: s→u=7, s→x=4, x→u=1, u→v=3. The labels above vertices are current d-values.

Frame 0 — initialization



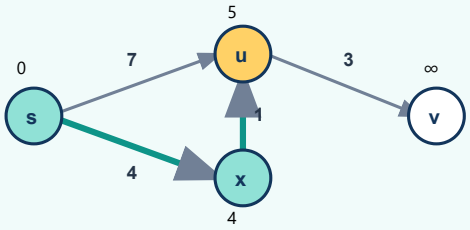
`d=[s:0,u:∞,x:∞,v:∞]`

Frame 1 — relax (s,u) and (s,x)



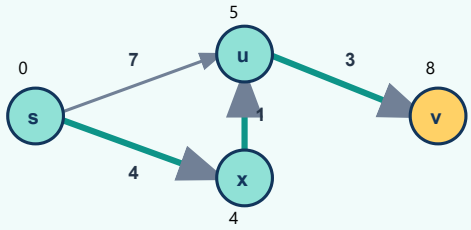
`u: ∞→7 via s; x: ∞→4 via s`

Frame 2 — relax (x,u): overwrite parent



`4+1=5<7 ⇒ d[u]=5, π[u]=x`

Frame 3 — relax (u,v); truth moves forward



`5+3=8<∞ ⇒ d[v]=8, π[v]=u`

What the pictures prove intuitively: a predecessor edge may be replaced when a cheaper route is discovered. Once a vertex receives its true shortest distance, relaxing the next edge of a shortest path propagates truth to the next vertex.

Why ordered relaxation works

Let $p=\langle v_0=s,v_1,\ldots,v_k=v\rangle$ be a shortest path. If its edges are relaxed in path order, correctness walks forward like falling dominoes.

After relaxing	What becomes permanently correct?	Reason
(v_0,v_1)	$d[v_1]=\delta(s,v_1)$	$d[s]=0=\delta(s,s)$, then convergence
(v_1,v_2)	$d[v_2]=\delta(s,v_2)$	$d[v_1]$ is correct, then convergence
...	...	induction
(v_{k-1},v_k)	$d[v]=\delta(s,v)$	the whole shortest path has propagated

Path-relaxation property. The result remains true even if unrelated relaxations occur between these edges. Upper bounds cannot fall below the true shortest distance, so extra work cannot destroy a correct label.

Predecessors and path reconstruction

```
PRINT-PATH(G, s, v)
if v == s
    print s
else if v.π == NIL
    print "no path"
else
    PRINT-PATH(G, s, v.π)
    print v
```

Direction alert: $\pi[v]$ points backward toward s. To print forward, recurse (or push vertices on a stack) before printing v.

Shortest paths and cycles

Cycle weight on a walk	Effect	Conclusion
positive	removing it makes the walk cheaper	not needed in a shortest path
zero	removing it preserves weight	a simple shortest path still exists
negative and reachable	repeating it decreases weight without bound	no finite shortest path downstream

Why at most $V-1$ edges? When no reachable negative cycle affects v, a shortest $s\to v$ path can be chosen simple. A simple path visits at most V vertices, hence uses at most $V-1$ edges. This is the exact fact Bellman–Ford exploits.

LECTURE 05

Bellman–Ford: negative edges handled correctly

Bellman–Ford repeatedly relaxes every edge. It returns TRUE if no negative-weight cycle is reachable from s; otherwise it returns FALSE.

```
BELLMAN-FORD(G, w, s)
INITIALIZE-SINGLE-SOURCE(G, s)
for i = 1 to |G.V| - 1
    for each edge (u,v) ∈ G.E
        RELAX(u,v,w)
for each edge (u,v) ∈ G.E
    if v.d > u.d + w(u,v)
        return FALSE                // reachable negative cycle
return TRUE
```

MAIN PHASE

$V-1$ full edge scans. After pass i, every shortest path using at most i edges has been accounted for.

DETECTION PHASE

One extra scan. Any further improvement proves a reachable negative cycle.

Why $V-1$ passes?

Choose a simple shortest path p with $k \le V-1$ edges. In each full scan, at least the next edge of p is relaxed after its predecessor has become correct (regardless of edge order, one full pass advances correctness by at least one path edge). Therefore after $V-1$ passes every reachable finite shortest distance is correct.

Complexity

$$\text{Time} = \Theta(VE) \qquad \text{Space} = \Theta(V) \text{ beyond graph storage}$$

There are $V-1$ scans plus one checking scan, each examining E edges. Initialization costs $\Theta(V)$.

Safe early exit: if an entire pass makes no update, stop early: all future passes would also make no update. Worst-case complexity remains $\Theta(VE)$.

What the return value does—and does not—say

- **TRUE:** no negative cycle is reachable from s; finite d-values are correct.
- **FALSE:** a negative cycle reachable from s exists.
- A negative cycle in an unreachable component does **not** make this source's run return FALSE, because all its vertices retain $d=\infty$.

Bellman–Ford dry run: the lecture-06 graph reconstructed

Edge scan order used by the available slides:

(t,x)=5, (t,y)=8, (t,z)=−4, (x,t)=−2, (y,x)=−3, (y,z)=9, (z,x)=7, (z,s)=2, (s,t)=6, (s,y)=7.

State	d[s]	d[t]	d[x]	d[y]	d[z]	Important changes in this pass
Initialize	0	∞	∞	∞	∞	source only
Pass 1	0	6	∞	7	∞	late edges s→t and s→y become reachable
Pass 2	0	6	4	7	2	x:11→4 via y; z:2 via t
Pass 3	0	2	4	7	2	t:2 via x
Pass 4	0	2	4	7	−2	z:−2 via t
Extra scan	no edge can improve					return TRUE

Final predecessor edges

Vertex	Distance	Predecessor	Shortest route
y	7	s	s→y
x	4	y	s→y→x
t	2	x	s→y→x→t
z	−2	t	s→y→x→t→z

The dry-run discipline: respect the stated edge order. Bellman–Ford's intermediate tables can differ under another order, although the final distances (when well-defined) do not.

Do not update all vertices "simultaneously" unless the question explicitly uses that variant. Standard CLRS pseudocode updates in place, so a later edge in the same pass can use an earlier update.

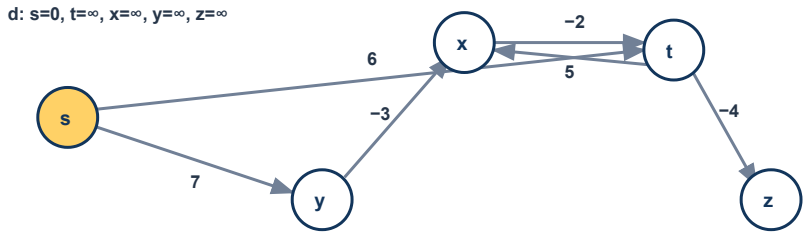
One update explained

During pass 2: $d[y]+w(y,x)=7+(-3)=4 < 11$, so set $d[x]=4$ and $\pi[x]=y$.

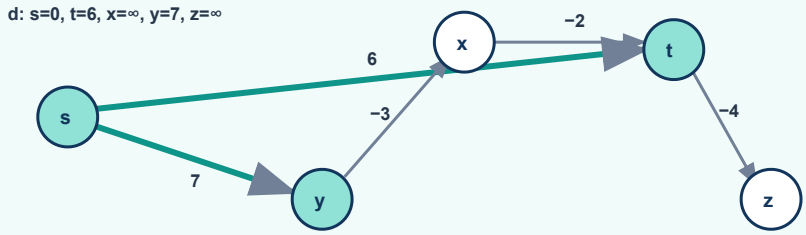
Bellman–Ford simulation I: distances spread from s

Green arrows show predecessor edges known at the end of the pass. The scan order places s's outgoing edges last, which is why pass 1 reaches only t and y.

Frame 0 — initialization

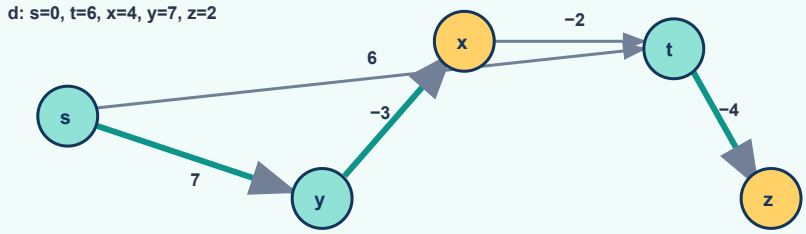


Frame 1 — after pass 1



Because (s,t) and (s,y) appear at the end of the scan, earlier edges cannot use these new values until pass 2.

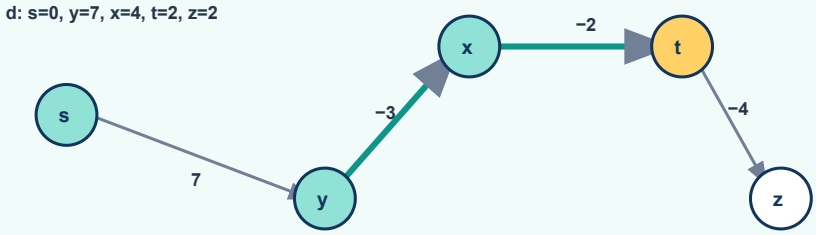
Frame 2 — after pass 2



t first proposes x=11; later y→x improves it to 4. Also t→z gives z=2.

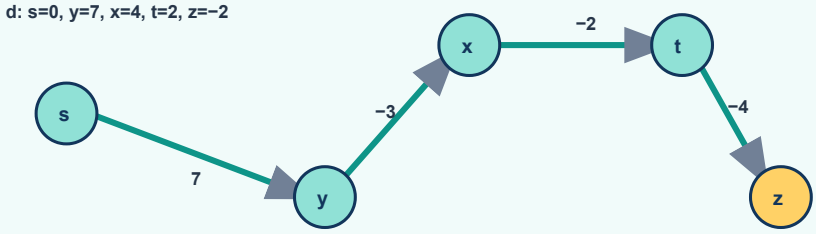
Bellman–Ford simulation II: the four-edge shortest path completes

Frame 3 — pass 3: x improves t



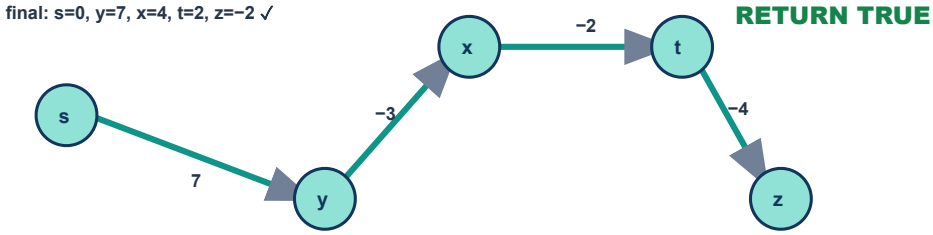
`x→t: 4+(−2)=2 < 6, so π[t]=x`

Frame 4 — pass 4: t improves z



`t→z: 2+(−4)=−2 < 2, so π[z]=t`

Frame 5 — extra scan: no improvement



`Shortest-path tree: s→y→x→t→z`

Path-length interpretation: the final route to z uses four edges. With this unfavorable scan order, its correct value appears only on pass 4—exactly illustrating why the worst case requires V−1 passes.

Negative-cycle detection and correctness

Why the extra pass detects a reachable negative cycle

If Bellman–Ford returns FALSE: some edge still satisfies $d[v] > d[u] + w(u,v)$. After $V-1$ passes this cannot happen when all reachable shortest distances are finite; therefore a reachable negative cycle must prevent convergence.

If a reachable negative cycle exists: suppose no cycle edge were relaxable after the main phase. Then for every cycle edge (v_i, v_{i+1}), $d[v_{i+1}] \le d[v_i] + w(v_i, v_{i+1})$. Summing cancels all d terms and implies $0 \le \text{weight(cycle)}$, contradicting that its weight is negative. Hence at least one edge relaxes.

Correctness theorem—answer skeleton

Theorem. If no negative-weight cycle is reachable from s, Bellman–Ford terminates with $d[v] = \delta(s,v)$ for every reachable v, and the predecessor subgraph is a shortest-path tree. **Typical 6–8 marks**

Proof: Fix reachable v and choose a simple shortest path p from s to v. It has at most $V-1$ edges. Across the $V-1$ full scans, the edges of p are relaxed in path order (at least one more per pass); by the path-relaxation property, $d[v] = \delta(s,v)$. Unreachable vertices remain ∞ by the no-path property. With all labels correct, the predecessor-subgraph property gives a shortest-path tree. Finally, triangle inequality ensures no edge can relax in the extra scan, so the algorithm returns TRUE.

Small negative-cycle example

$s \to a = 1$, $a \to b = 2$, $b \to c = -5$, $c \to a = 1$. The cycle $a \to b \to c \to a$ weighs $2 - 5 + 1 = -2$.

After each trip around cycle	Best known cost to a
before cycling	1
once	−1
twice	−3
k times	$1 - 2k \to -\infty$

Therefore "shortest" is undefined as a finite value. For any claimed route, one more traversal gives a cheaper one.

Choosing the right algorithm

Problem / restriction	Algorithm	Time	Key reason
Undirected, connect all vertices cheaply	Prim or Kruskal	typically O(E log V)	MST, not path distance
SSSP, arbitrary directed graph, negative edges possible	Bellman–Ford	Θ(VE)	detects reachable negative cycles
SSSP, DAG, even with negative edges	DAG shortest paths	Θ(V+E)	topological order; no cycles
SSSP, all edge weights nonnegative	Dijkstra	O((V+E)log V) typical	greedy finalization is safe
Unweighted graph / equal edge weights	BFS	Θ(V+E)	layers equal number of edges

Prim and Dijkstra look alike—but are not alike

Question	Prim	Dijkstra
What does key mean?	cheapest one edge connecting v to current tree	cheapest total source-to-v path found
Update	if w(u,v)<key[v]	if d[u]+w(u,v)<d[v]
Output objective	minimum total spanning-tree weight	minimum source-to-each-vertex path weights
Negative edges	fine	break greedy finalization

> **Algorithm decision mnemonic:** Connect everyone → MST. Paths with no weights → BFS. DAG → topological relaxation. Negative edges → Bellman–Ford. Nonnegative weights → Dijkstra.

High-frequency comparison: Prim vs Bellman–Ford

	Prim	Bellman–Ford
Graph	weighted undirected	weighted directed (or converted undirected)
Goal	minimum spanning tree	single-source shortest paths
Core operation	extract minimum key; update attachment edges	relax every edge repeatedly
Proof tool	cut property / safe edge	path-relaxation + simple path ≤V−1 edges

Full-mark answer templates

“Explain Prim’s algorithm with correctness and complexity”

1. State input assumptions: connected, weighted, undirected graph.
2. Define key, π, Q, and the greedy choice.
3. Write pseudocode with the Q-membership test.
4. Give a small extraction/key-update table.
5. Prove safety using cut $(V-Q, Q)$.
6. State complexity for the requested representation.

“Explain Bellman–Ford and prove correctness”

1. State that negative edges are permitted and reachable negative cycles are detected.
2. Write initialization, $V-1$ edge scans, and the extra scan.
3. Explain relaxation and predecessor changes.
4. Use “simple shortest path has $\leq V-1$ edges.”
5. Invoke path-relaxation property for correctness.
6. Use the summed-cycle-inequalities argument for detection.
7. State $\Theta(VE)$ time and $\Theta(V)$ auxiliary space.

“What is relaxation?”

> Relaxation tests whether reaching v through u improves the current upper bound. If $d[v] > d[u] + w(u,v)$, assign $d[v] = d[u] + w(u,v)$ and $\pi[v] = u$. Repeated relaxation propagates correct shortest-path information along paths.

Mark-saving habits

- □ Write the graph restriction before choosing an algorithm.
- □ Distinguish ∞ (unreachable) from $-\infty$ (unbounded below).
- □ For MST, report $V-1$ chosen edges and their total weight.
- □ For SSSP, report distances and predecessors.
- □ In proofs, name the theorem/property you use.
- □ In dry runs, show failed relaxations only when they explain a key point; always show successful changes.

Practice set with compact solutions

1. Can Prim work with negative edge weights?
Yes. Its cut-property proof requires only weight comparisons. Cycles cannot be repeated because the output is a tree.

2. Why is the condition "v∈Q" essential in Prim?
Without it, the algorithm might change the parent/key of a vertex already fixed inside the tree, corrupting the chosen MST structure.

3. Is every edge selected by Prim globally lightest among all unused edges?
No. It is lightest among edges crossing the current cut $(V-Q, Q)$.

4. If Bellman–Ford makes no update on pass 3, must it still do later passes?
No. A full unchanged scan is a fixed point; later identical scans cannot change any label.

5. Why does an unreachable negative cycle not trigger detection?
Its vertices have $d=\infty$. For its edges, ∞ is not greater than $\infty+w$ under the mathematical convention, so no relaxation occurs.

6. Does a negative edge imply a negative cycle?
No. A cycle needs a closed directed route whose total weight is negative.

7. Give a graph where Dijkstra fails but Bellman–Ford succeeds.
$s\to a=2$, $s\to b=5$, $b\to a=-10$. Dijkstra may finalize a at 2 before discovering $s\to b\to a=-5$; Bellman–Ford permits the later correction.

8. Prove a subpath of a shortest path is shortest.
If the subpath were replaceable by a cheaper one, replacing it would make the whole path cheaper, a contradiction.

9. What differs when Bellman–Ford's edge order changes?
Intermediate pass values and how quickly they converge can differ. Final finite distances and negative-cycle verdict do not.

10. Can a predecessor tree be unique when shortest distances are unique?
Distances are scalar values and may be unique while several equal-weight paths realize them; the predecessor tree need not be unique.

Final two-page revision wall

Prim

- Connected weighted undirected graph.
- Grow one tree from arbitrary root.
- key[v]=cheapest crossing edge to v.
- Extract min key; update outside neighbors.
- Proof: cut respects A; light edge is safe.
- Matrix $\Theta(V^2)$; binary heap O(E log V).

Relaxation

$$\text{if } d[v] > d[u]+w(u,v):$$
$$d[v] \leftarrow d[u]+w(u,v),\ \pi[v] \leftarrow u$$

- d is an upper bound.
- Correctness propagates along shortest-path edges.
- π stores the actual route.

Bellman–Ford

- Negative directed edges allowed.
- Initialize d[s]=0; all others ∞.
- Relax all E edges for V−1 passes.
- One extra scan detects a reachable negative cycle.
- Proof: simple shortest path ≤V−1 edges + path relaxation.
- Time Θ(VE); auxiliary space Θ(V).

Distance meanings

- finite: shortest path exists
- ∞: unreachable
- −∞: reachable negative cycle can lead to vertex

Proof map

Claim	Tool	One decisive sentence
Prim is correct	cut property	minimum key gives a light edge across (V−Q,Q)
Relaxation reaches truth	convergence	correct u plus shortest-path edge makes v correct
Bellman–Ford is correct	path relaxation	V−1 passes cover every edge of a simple shortest path
Negative cycle detected	sum inequalities	if no cycle edge relaxed, summing would imply cycle weight ≥0

Last-minute rule: Do not memorize only pseudocode. For every algorithm know: input restriction, state meaning, greedy/relaxation step, invariant, proof tool, output, and complexity.

Final self-test

Close the note and answer aloud. If any answer takes longer than 30 seconds, revisit the named section.

1. What exactly does key[v] mean in Prim?
2. Which cut proves Prim's chosen edge safe?
3. Why is Prim not Dijkstra?
4. State relaxation without looking.
5. What are the three possible meanings of $\delta(s,v)$?
6. Why can a shortest path be assumed to have at most $V-1$ edges?
7. Why does Bellman–Ford need $V-1$ passes?
8. Why does an extra successful relaxation prove a negative cycle?
9. Why must that negative cycle be reachable from s?
10. State Bellman–Ford time and auxiliary space.
11. What changes if edge order changes?
12. How do you reconstruct a path from π?

> **Professor's closing advice:** In an exam, a clean invariant and a labelled dry-run table beat a page of vague narration. Show the state, show the legal update, and name the theorem that makes it safe.

Source alignment

Primary local reference: Cormen, Leiserson, Rivest and Stein, *Introduction to Algorithms*, 3rd ed., §23.1–23.2 and Chapter 24 introduction, §24.1, and shortest-path properties in §24.5. Lecture placement was inferred from the supplied CSE 207 Lecture 01, 02, 06, 07 and 08 files. Diagrams and wording in this note are original teaching material.